

Dokumentacja techniczna

Wizualizacja 3D modelu atomu - Grafika Komputerowa

1. Cel projektu

Celem projektu jest interaktywna wizualizacja modelu atomu w OpenGL 3.3. Scena zawiera jadro złożone z 12 kul (protony i neutrony), trzy orbity elektronów z animacją ruchu, podłogę oraz dynamiczne oświetlenie Phong. Użytkownik może sterować kamerą, obrotem i skalą modelu oraz trybem filtrowania tekstur.

2. Architektura plików

src/main.cpp - główna pętla aplikacji, generowanie geometrii, tekstury, input.

include/Camera.h - kamera perspektywiczna (WASD, mysz, scroll).

include/Shader.h - kompilacja i użycie shaderów GLSL.

shaders/vertex_shader.vs, fragment_shader.fs - cieniowanie Phong + tekstury.

shaders/line_vertex.vs, line_fragment.fs - rysowanie orbit (linie).

textures/nucleus.png, floor.png - mapy diffuse.

external/glad/ - loader OpenGL; external/stb/stb_image.h - ładowanie obrazów.

CMakeLists.txt - konfiguracja build (GLFW, GLM, GLAD).

3. Opis sceny

Modele 3D (min. 3): sfera (generateSphere), szescian (generateCube - podłoga), okrag (generateOrbit - linie orbit).

Oświetlenie: point light z ambient (0.15), diffuse i specular (Phong, shininess=32). Pozycja światła animowana - orbita wokół sceny.

Animacja: obrót jadra, ruch elektronów po orbitach z różną prędkością, ruch źródła światła.

Teksturowanie: nucleus.png na protonach, floor.png na podłożu.

4. Sterowanie

W/A/S/D - ruch kamery do przodu/tyłu/lewo/prawo

Mysz - obrót kamery (look-around)

Scroll - zoom (FOV)

Q / E - obrót całego modelu atomu wokół osi Y

R / F - skalowanie modelu atomu (0.3 - 2.5)

F1 - filtrowanie tekstur GL_NEAREST

F2 - filtrowanie GL_LINEAR

F3 - filtrowanie GL_LINEAR_MIPMAP_LINEAR

ESC - zamknięcie aplikacji

5. Shadery

Vertex shader: transformacja MVP, przekazanie FragPos, Normal i TexCoords.

Fragment shader: model Phong (ambient + diffuse + specular) z opcjonalną teksturą (uniform useTexture, sampler2D diffuseMap).

Line shader: prosty kolor linii bez oświetlenia - unika błędów przy rysowaniu orbit.

6. Filtrowanie tekstur

Tekstury ladowane przez stb_image. Tryb filtrowania ustawiany przez glTexParameteri (MIN/MAG_FILTER). Mipmap generowany przez glGenerateMipmap. Przełączanie F1/F2/F3 pozwala porównać NEAREST, LINEAR i MIPMAP w czasie rzeczywistym.

7. Kompilacja i uruchomienie

Wymagania: CMake 3.16+, kompilator C++17, GLFW 3, GLM.

macOS: brew install cmake glfw glm

```
mkdir build && cd build
```

```
cmake ..
```

```
cmake --build .
```

```
./atom
```

Shadery i tekstury kopiowane automatycznie do katalogu build przy kompilacji.

8. Mapowanie na wytyczne projektu

1. Inicjalizacja OpenGL/GLFW - main(), gladLoadGLLoader
2. Min. 3 modele 3D - sfera, szescian, orbita
3. Kamera perspektywiczna - Camera.h, glm::perspective
4. Interakcja - kamera + obrot/skalowanie obiektów (Q/E/R/F)
5. Oświetlenie - ambient + point light
6. Cieniowanie Phong'a - fragment_shader.fs
7. Teksturowanie - nucleus.png, floor.png
8. Animacja - elektrony, jadro, światło
9. Komentarze w kodzie + niniejsza dokumentacja PDF
10. Filtrowanie obrazu - F1/F2/F3, GL_NEAREST/LINEAR/MIPMAP